```
$concat = $hello . $space . $world . $exclamation ;
var_dump($concat);
var_dump(bin2hex($concat));
```

```
Output:
string(12) "Hello World!"
string(24) "48656c6c6f20576f726c6421"
```

The string concatenation operator – dot `.` – is just joining four buffers together. To drive the point home, the example above mixes string literal values with byte escapes, the syntax of which is `\x` followed by an 8-bit hexadecimal value and enclosed in double quotes. `"\x20"` is therefore a single byte with a decimal value of *32*. The codepages for ASCII, ISO-8859-1, and Unicode/UTF-8 all agree that *0x20* is a space character, and *0x21* is the Latin exclamation punctuation mark. The output of `bin2hex()` shows us the values of each byte in the new `$concat` buffer: `48656c6c...` is the first four bytes of the sequence: *0x48 0x65 0x6c 0x6c* or `H e l l`... you can work out the rest.

Here's a slightly tweaked version that uses Unicode escapes instead of UTF-8 bytes to demonstrate that Unicode is a true superset of ASCII:

```
$hello = "Hello";
$space = "\u{20}";
$world = "World";
$exclamation = "\u{21}";
// concatenate and var_dump() as before...
```

```
Output:
string(12) "Hello World!"
```

The use of Unicode codepoint escapes *does* require some encoding awareness on PHP's part, in that it will internally have to calculate the UTF-8 code unit bytes that correspond to the codepoints before assignment, but the resulting data is still a byte stream and nothing more. The previous section demonstrated that Bald Man was a multi-codepoint character in Unicode; let's take a look at the byte stream PHP generated when creating the Bald Man buffer:

```
$baldMan = "\u{1F468}\u{200D}\u{1F9B2}";

var_dump($baldMan);
var_dump(bin2hex($baldMan));
```